

Docker Compose Guide: Run Multi-Container Apps Easily

Running modern software usually requires coordinating web servers, databases, and caches all at once. Docker Compose lets you define and manage your entire multi-container stack using a single text file.

What you'll learn:

1. Defining Your Application Blueprint
2. Launching Services in the Background
3. Troubleshooting with Live Log Streams
4. Stopping Services and Cleaning Up

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Defining Your Application Blueprint

Instead of typing endless docker run commands with long flags, you define your entire application setup inside a docker-compose.yml file. This file organizes images, open ports, and environment settings in a single clean format. You can validate syntax quickly using the config check tool.

```
$docker compose config
```

Cloud & DevOps Daily

Docker Compose: Run Your Whole Stack With One Command

Sunday, August 30, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Launching Services in the Background

To start your entire software stack, run the up command with the detached flag. Docker Compose creates private networks automatically so your web server can immediately talk to your database. Your terminal stays free while all services spin up smoothly in the background.

```
$ docker compose up -d
```

Cloud & DevOps Daily

Docker Compose: Run Your Whole Stack With One Command

Sunday, August 30, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Troubleshooting with Live Log Streams

When an application breaks, checking logs for individual containers takes too much time. Docker Compose merges log output from every running service into one live view. The `follow` flag keeps the logs open so you can trace user requests across services in real time.

```
$docker compose logs -f
```

Cloud & DevOps Daily

Docker Compose: Run Your Whole Stack With One Command

Sunday, August 30, 2026

Swipe to tip 4 >> [linkedin.com/in/mramanrajsingh](https://www.linkedin.com/in/mramanrajsingh)

4

TIP 4 OF 4

4. Stopping Services and Cleaning Up

Leaving inactive development containers running drains your laptop battery and consumes valuable system memory. The down command stops all active services and removes container networks cleanly. Adding the volume flag wipes temporary database storage when you need a fresh start.

```
$ docker compose down -v
```


Cloud & DevOps Daily

Docker Compose: Run Your Whole Stack With One Command

Sunday, August 30, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Store sensitive passwords in a hidden `.env` file instead of hardcoding them into YAML.
- + Use explicit image tags like `postgres:15` rather than using unpredictable latest tags.
- + Name your services simply like `web` or `db` so they are easy to reference inside code.
- + Run `docker compose config` before deploying changes to catch silent formatting mistakes.
- + Use `docker compose down -v` only when you genuinely want to delete local database data.

Watch Out For

- ! Mixing up spaces and tabs in YAML formatting, which causes immediate file parse errors.
- ! Hardcoding IP addresses inside code instead of using Compose service names like `db:5432`.
- ! Forgetting to use the `--build` flag after editing source code, running outdated containers.
- ! Exposing internal database ports to the internet host instead of keeping them isolated.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh